

Cours

Système d'Exploitation 2 (Operating System)

Enseignante : Yomna BEN JMAA

yomna.benjmaa@isimg.tn

2^{ème} année CPI

Année universitaire: 2025-2026

Objectifs

- Introduire les mécanismes de base utilisés par les systèmes pour gérer les processus et les ressources.
- Introduire les notions de Processus, Threads et Ressources.
- Présenter les techniques de gestion des processus.
- Présenter les techniques de gestion des ressources.
- Apprendre les techniques de sécurisation des systèmes et les techniques de protection des données

Plan

- 1 Mécanismes de base des Systèmes d'Exploitation
- 2 Gestion des processus
 - Processus & Thread
 - Synchronisation interprocessus
- 3 Gestion des ressources (Processeur, Mémoire centrale, Disque)
- 4 Mémoire virtuelle
- 5 Installation et paramétrage de systèmes Windows et Unix en mode virtuel et non virtuel

Chapitre 1

Mécanismes de base des Systèmes d'Exploitation

Enseignante : Yomna BEN JMAA

yomna.benjmaa@isimg.tn

Année universitaire: 2025-2026

Plan

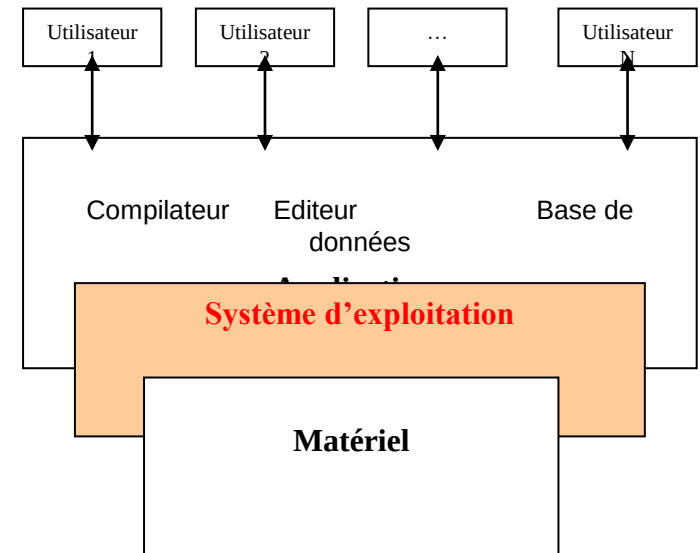
- **Introduction**
- **Composants d'un système d'exploitation**
- **Fonctionnalités des systèmes d'exploitation**
- **Caractéristiques des systèmes d'exploitation**
- **Architecture d'un Système d'exploitation**
- **Exemples de systèmes d'exploitation**

Introduction

Système informatique:

L'objectif d'un système informatique est d'automatiser le traitement de l'information. Il se décompose généralement en :

- **Matériel** : l'unité centrale (UC), la mémoire et les périphériques d'entrée/sortie. Il fournit les ressources informatiques de bases : *la machine physique*
- **Logiciel** : les programmes exécutés sur un ordinateur sont classés selon leurs fonctions :
 - Logiciels d'applications (les programmes bureautique)
 - Logiciels de base (**système d'exploitation**)

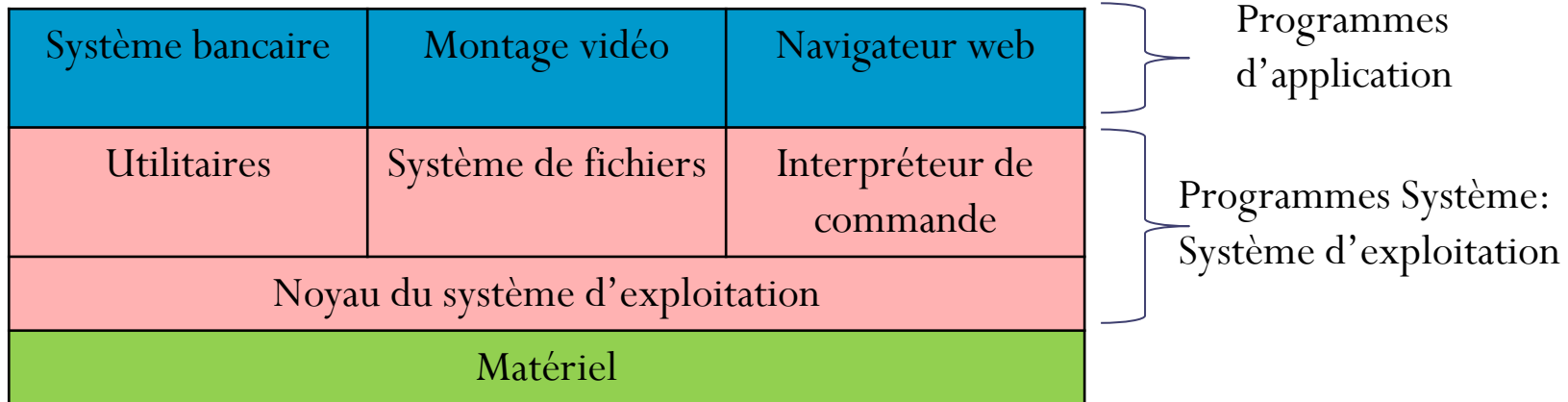


Introduction

Système d'exploitation:

- Le programme fondamental de tous les programmes systèmes ; c'est l'ensemble des programmes qui se chargent de résoudre les problèmes relatifs à l'exploitation de l'ordinateur.
- Le programme qui gère les ressources physiques de l'ordinateur ; il assure l'exploitation efficace, fiable et économique des ressources critiques et/ou partagées (processeurs, mémoire)
- Le programme qui gère l'interaction avec l'utilisateur, il facilite son travail en lui présentant une machine plus simple à exploiter que la machine réelle.

Composants d'un système d'exploitation



- **Noyau (kernel)** : Cœur du système, gère la communication matériel/logiciel et se charge en mémoire au démarrage.
- **L'interpréteur de commandes (shell)** : assure le dialogue entre le système d'exploitation et l'utilisateur. Il analyse les commandes de l'utilisateur afin d'exécuter les actions demandées. Il peut être en mode texte ou en mode graphique.
- **Les utilitaires** : un ensemble de programmes nécessaires à l'exploitation de base de l'ordinateur (exemples : éditeurs de texte, visualiseurs d'images, lecteurs des fichiers multimédia)
- **Le système de fichiers (File System)** : Organise les données en arborescence, gère création, lecture, modification et suppression des fichiers.

Systeme d'exploitation: Fonctionnalités (1 / 4)

Les principales fonctions d'un SE sont reliées aux différentes ressources manipulées par l'ordinateur. On distingue :

1. Gestion des processus

- Un processus est un programme en cours d'exécution sur CPU,
- Un processus a besoin de certaines ressources : CPU, mémoire, fichiers, les périphériques d'E/S, ... pour accomplir sa tâche
- Le SE est responsable de :
 - Création et suppression de processus
 - Suspension et relancement du processus
 - Mécanismes pour synchronisation, communication entre processus et traitement des interblocages

Systeme d'exploitation: Fonctionnalités (2/4)

2. Gestion de la mémoire

- La mémoire est un espace de stockage de données, rapidement accessible par le CPU et les périphériques d'E/S.
- Le SE est responsable de :
 - Décider quels processus à charger en mémoire dès que l'espace mémoire devient disponible
 - Allouer et libérer la mémoire sous la demande des processus

Systeme d'exploitation: Fonctionnalités (3/4)

3. Gestion des fichiers

Le SE est responsable des activités suivantes :

- Création et suppression de fichiers
- Création et suppression de répertoires
- Mapper les fichiers en mémoire secondaire
- Backup de fichiers sur un média de stockage non volatil

Système d'exploitation: Fonctionnalités (4/4)

4. Gestion des périphériques (Entrées / Sorties)

Configuration, Accès et utilisations des différents périphériques.

5. Gestion de l'interface Homme / Machines

Fournir une interface conviviale, transparente et facile à utiliser

6. Gestion du réseau

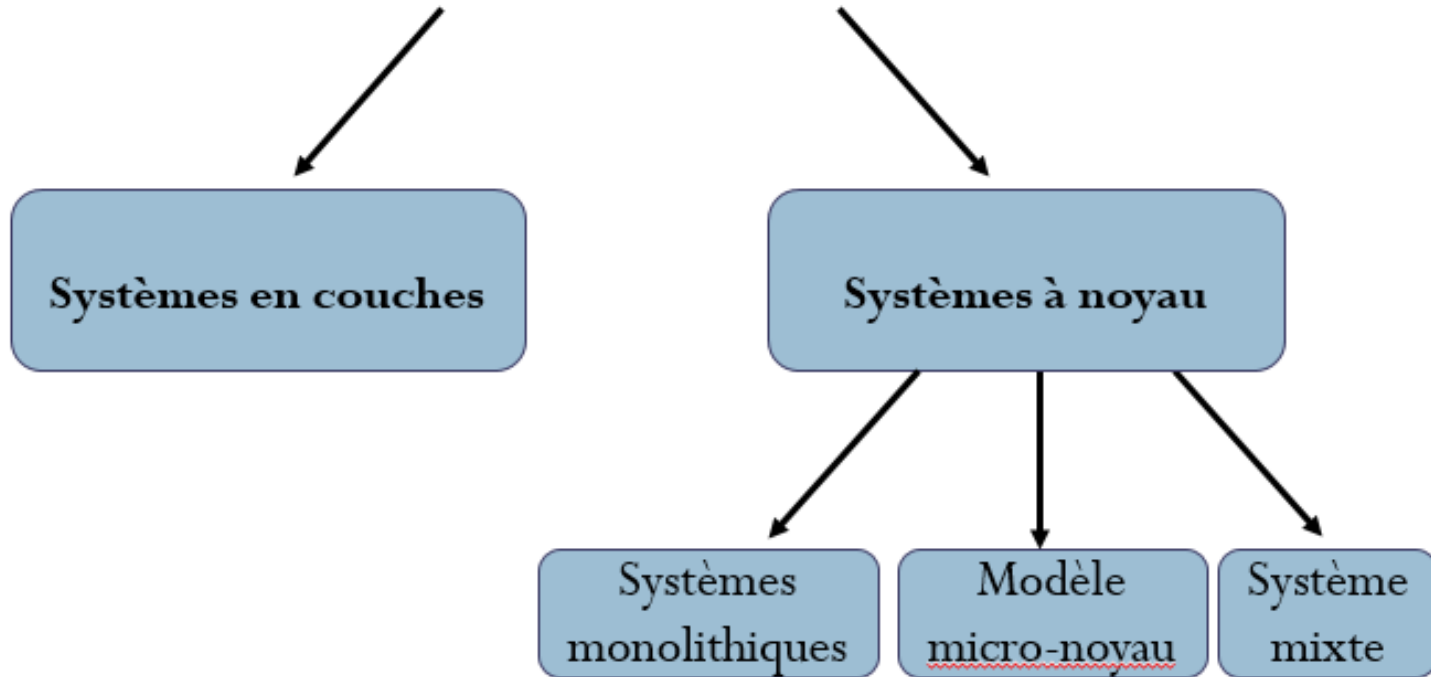
L'interconnexion des machines par des réseaux locaux, offre un partage des fichiers utilisateurs, une protection pour ces fichiers, l'identification des machines et des utilisateurs connectés au réseau.

Systeme d'exploitation : Caracteristiques

Le SE doit assurer :

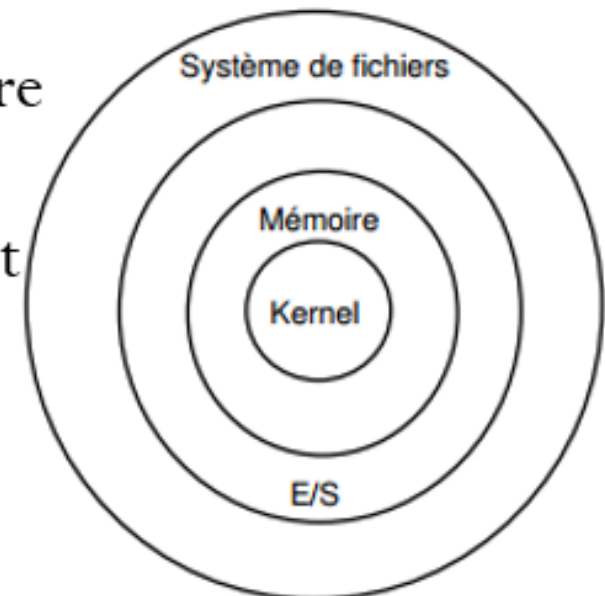
- **Fiabilité** : limiter les conséquences des défaillances matérielles ou des erreurs des utilisateurs. En cas de panne, éviter les pertes d'information ou leur incohérence.
- **Efficacité** : Utiliser au mieux les ressources
- **Equité** (pas de programme en attente indéfinie)
- **Cohérence (entre les accès consécutifs)** : garantir la logique des accès successifs aux données
- **Facilité d'utilisation** : Offrir un langage de commande (dialogue usager/système) et des diagnostics d'erreurs (système/usager) clairs et précis
- **Adaptabilité** : permettre des modifications matérielles et logicielles les plus simples possibles, à l'aide d'outils spécialisés
- **Protection contre les accès interdits**

Structures des Systèmes d'exploitation



Structure en couches (1/2)

- Systèmes **organisés par couches**
- Une communication se crée entre couches adjacentes.
- Avantage : les couches sont totalement indépendantes les unes des autres.
- Inconvénient : très complexe à mettre en œuvre et les systèmes ainsi structurés sont souvent très lourds et peu performants.

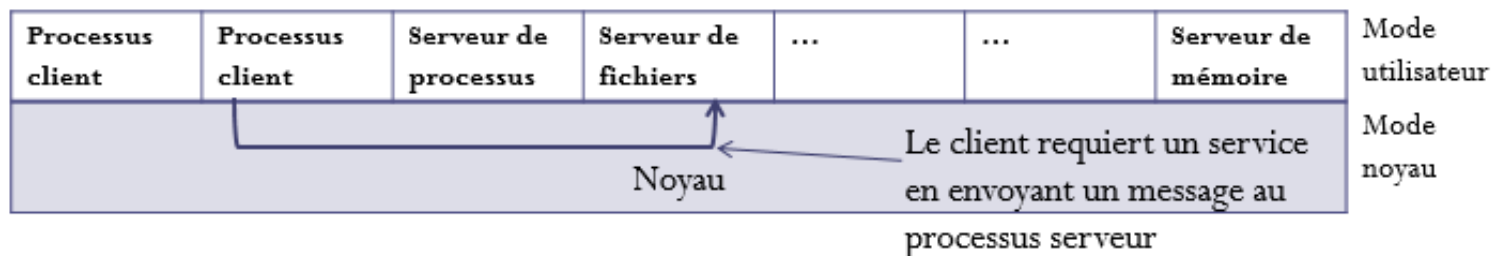


Systèmes monolithiques

- Couche logicielle entre les applications et le matériel. Il fonctionne sur deux espaces :
 - **L'espace noyau** : constitué des programmes qui fournissent des services aux applications (ex. affichage des messages, utilisation de l'imprimante, gestion de la souris, ...)
 - **L'espace utilisateur** : contient les applications et les données. En effet, une application est un programme réalisant un objectif en s'appuyant sur les services de l'espace noyau du système.
- Exemple : les premières versions d'Unix, MS-DOS

Modèle micro-noyau

- Système à un noyau minimal (appelé **micro-noyau**) de taille de code réduite.
- La lecture d'un bloc d'un fichier se fait lorsque le processus client envoie une requête à un processus serveur qui fait leur travail et renvoie une réponse.



- Le noyau gère la communication entre les clients et les serveurs.
- Inconvénient : La couche de dialogue ralentit les opérations et conduit à une baisse de performance.

Les classes des systèmes d'exploitation

- Il existe différentes classes de systèmes d'exploitation selon :
 - Les services rendus
 - L'architecture logique
 - L'architecture matérielle

Selon les services rendus (1/2)

- Systèmes mono ou multi-tâches: C'est la capacité du système à pouvoir exécuter un ou plusieurs processus à la fois.
- Système mono-tâche : ce système n'autorise l'allocation du processeur que pour une seule tâche. En effet, une tâche ne prend le processeur que si et seulement si la précédente est achevée, si non le système se bloque.
- Système mutli-tâches : capacité du système à pouvoir exécuter plusieurs tâches «processus » simultanément.

Selon les services rendus (2/2)

- Systèmes mono ou multi-utilisateurs : C'est la capacité ou pas de gérer un nombre d'utilisateurs utilisant simultanément les mêmes ressources matérielles
 - Mono-utilisateur : le système ne peut gérer qu'un seul utilisateur (ex : le système MS-DOS)
 - Multi-utilisateurs : capacité à pouvoir gérer un panel d'utilisateurs utilisant simultanément les mêmes ressources matérielles.

Selon l'architecture logique

- Système centralisé: L'ensemble du système est entièrement présent sur la machine. Le système ne gère que les ressources de la machine sur laquelle il est présent.
- Système réparti ou distribué: Les différentes abstractions du système sont réparties sur un ensemble de machines. Exemples : Inferno, MOSIX, Plan9, etc.
 - Le système d'exploitation réparti apparaît aux yeux de ses utilisateurs comme une machine virtuelle unique monoprocesseur.
 - L'utilisateur ne se soucie pas de la localisation des ressources.
 - Ces systèmes offrent des solutions de résistance aux pannes.

Selon l'architecture matérielle (1/2)

- **Architecture monoprocesseur** : Temps partagé ou multiprogrammation
 - Ressource processeur unique : Il a fallu développer un mécanisme de gestion des processus pour offrir un (pseudo) parallélisme à l'utilisateur : c'est le temps partagé. Il s'agit en fait d'une commutation rapide entre les différents processus pour donner l'illusion d'un parallélisme.
- **Architecture multiprocesseur** :
 - Le multiprocessing est une technique consistant à faire fonctionner plusieurs processeurs en parallèle afin d'obtenir une puissance de calcul plus importante que celle obtenue avec un processeur haut de gamme ou bien afin d'augmenter la disponibilité du système (en cas de panne d'un processeur):
 - Parallélisme

Application

Trois travaux T1, T2 et T3 possèdent les caractéristiques suivantes :

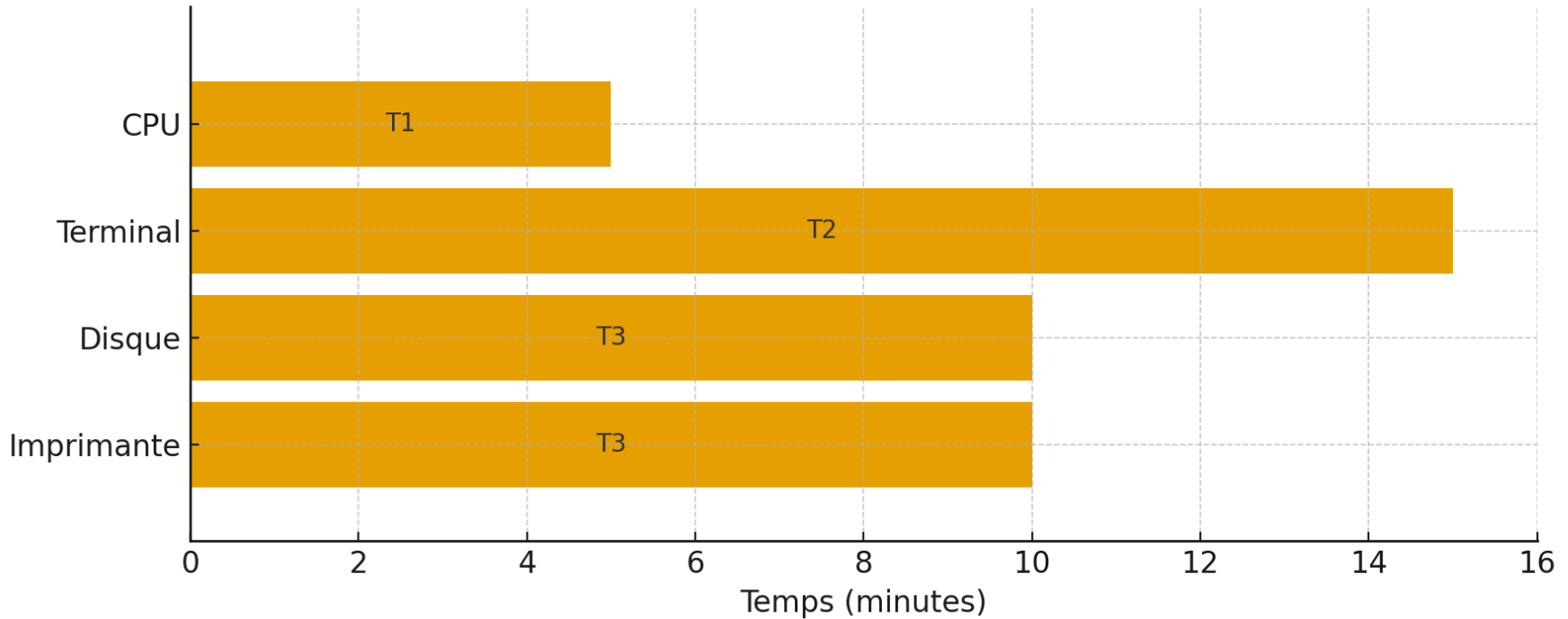
Ces travaux seront exécutés sur un ordinateur ayant une mémoire de 256K mots disponibles (partie non utilisée par le S.E.), un disque, un terminal et une imprimante.

	T1	T2	T3
Type de travail	Calcul	entrée/sortie	entrée/sortie
Durée	5 min	15 min	10 min
Besoin mémoire	50K	100K	80K
Besoin disque	non	non	oui
Besoin terminal	non	oui	non
Besoin imprimante	non	non	oui

Expliquer pourquoi la multiprogrammation est possible pour les travaux donnés.

Correction

Multiprogrammation : utilisation simultanée des ressources par T1, T2, T3



Chapitre 2

Gestion de processus

- Processus et thread
- Synchronisation interprocessus

Enseignante : Yomna BEN JMAA

yomna.benjmaa@isimg.tn

2^{ème} année CPI

Année universitaire: 2025-2026

Plan

- **Processus**
- **Threads**
- **Synchronisation interprocessus**

Notion de base des processus

- Un processus est une entité dynamique représentant un programme en cours d'exécution. Il dispose de ses propres ressources physiques (mémoire, processeur, entrée/sortie, ...) et logiques (données, variables, fichiers, ...)
- **Ne pas confondre un processus avec un programme**
 - Un programme est une suite d'instructions ; c'est du texte, un code statique.
 - Le processus est un concept dynamique, il représente le déroulement d'une tâche faisant partie d'une application ou un programme système quelconque.
- Si un système informatique ne comporte qu'un seul processeur, alors, à un instant donné, un seul processus aura accès à ce processeur.

Notions de bases des processus

- On peut trouver plusieurs processus simultanément en cours d'exécution (multiprogrammation et temps partagé)
- Le SE manipule deux familles de processus
 - **Processus système** : processus lancé par le SE (init processus père de tous les processus du système)
 - **Processus utilisateur** : processus lancée par l'utilisateur (commande utilisateur)

.

Structure des processus

- Les processus des utilisateurs sont lancés par un interprète de commande. Ils peuvent eux-mêmes lancer ensuite d'autres processus
- On appelle le processus créateur, le père, et les processus créés, les fils
- Les processus peuvent donc se structurer sous la forme d'une arborescence.
- Au lancement du système, il n'existe qu'un seul processus, qui est l'ancêtre de tous les autres.

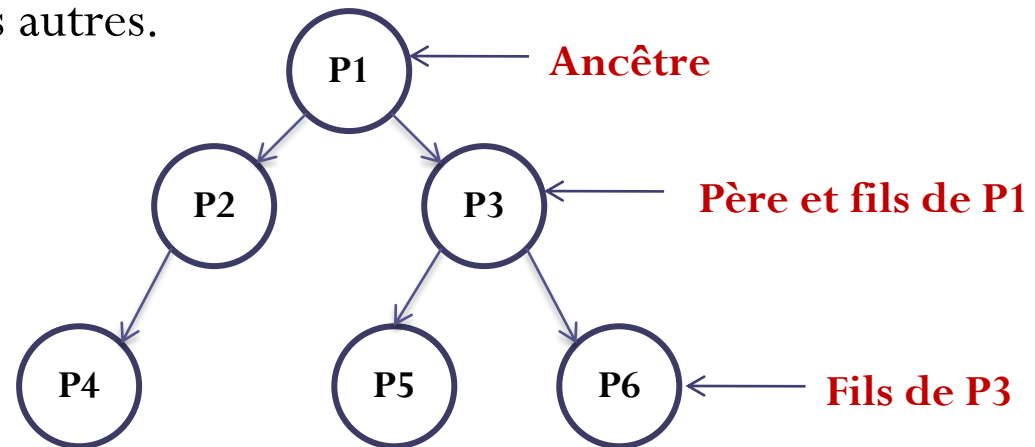
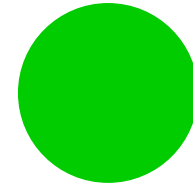


Figure1: Le hiérarchie des processus

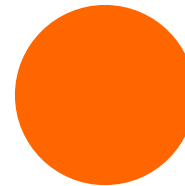
Etats d'un processus (1/3)

- Lors de l'exécution, processus change d'états;
- Trois états sont possibles pour un processus:
 - **actif (Elu)** quand le processeur est en train d'exécuter une de ses instructions.
 - Dans un système monoprocesseur, un seul processus peut être actif à la fois;
 - **bloqué** quand il est en attente d'un événement ou d'une synchronisation,
 - i.e. la fin d'une entrée-sortie ou plus généralement en attente d'un autre processus.
 - **prêt** s'il n'est ni actif ni bloqué.

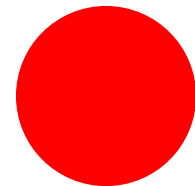
Actif
En cours d'exécution



Prêt



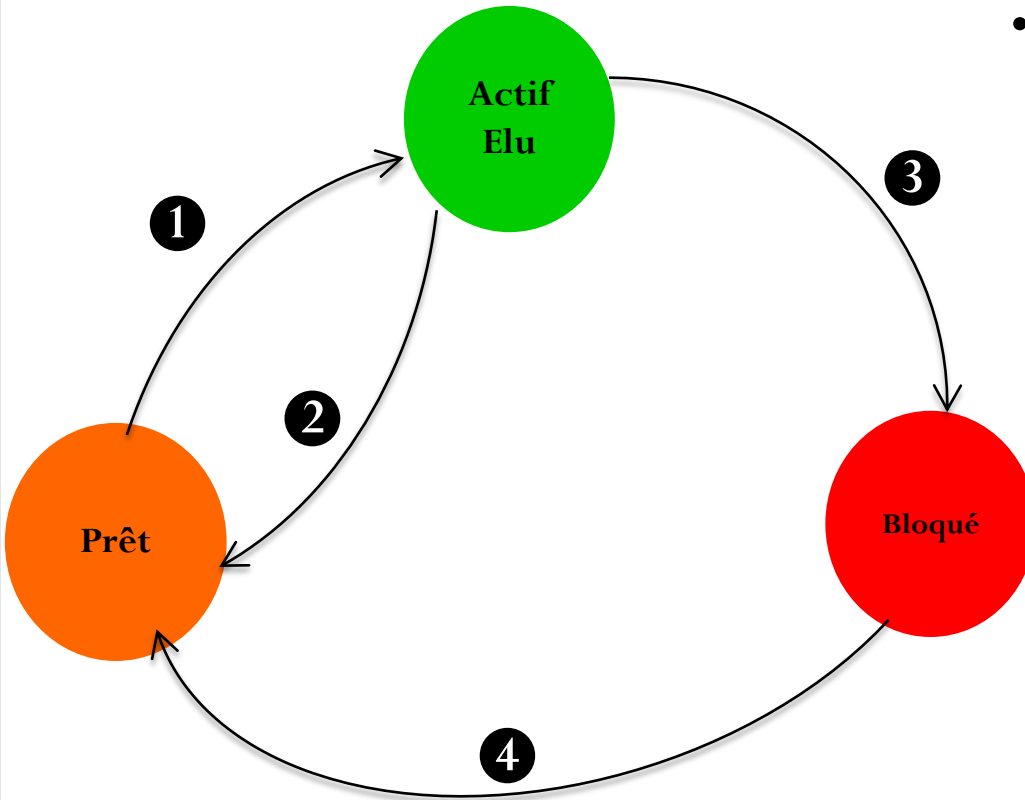
Bloqué



Attente d'un événement
extérieur

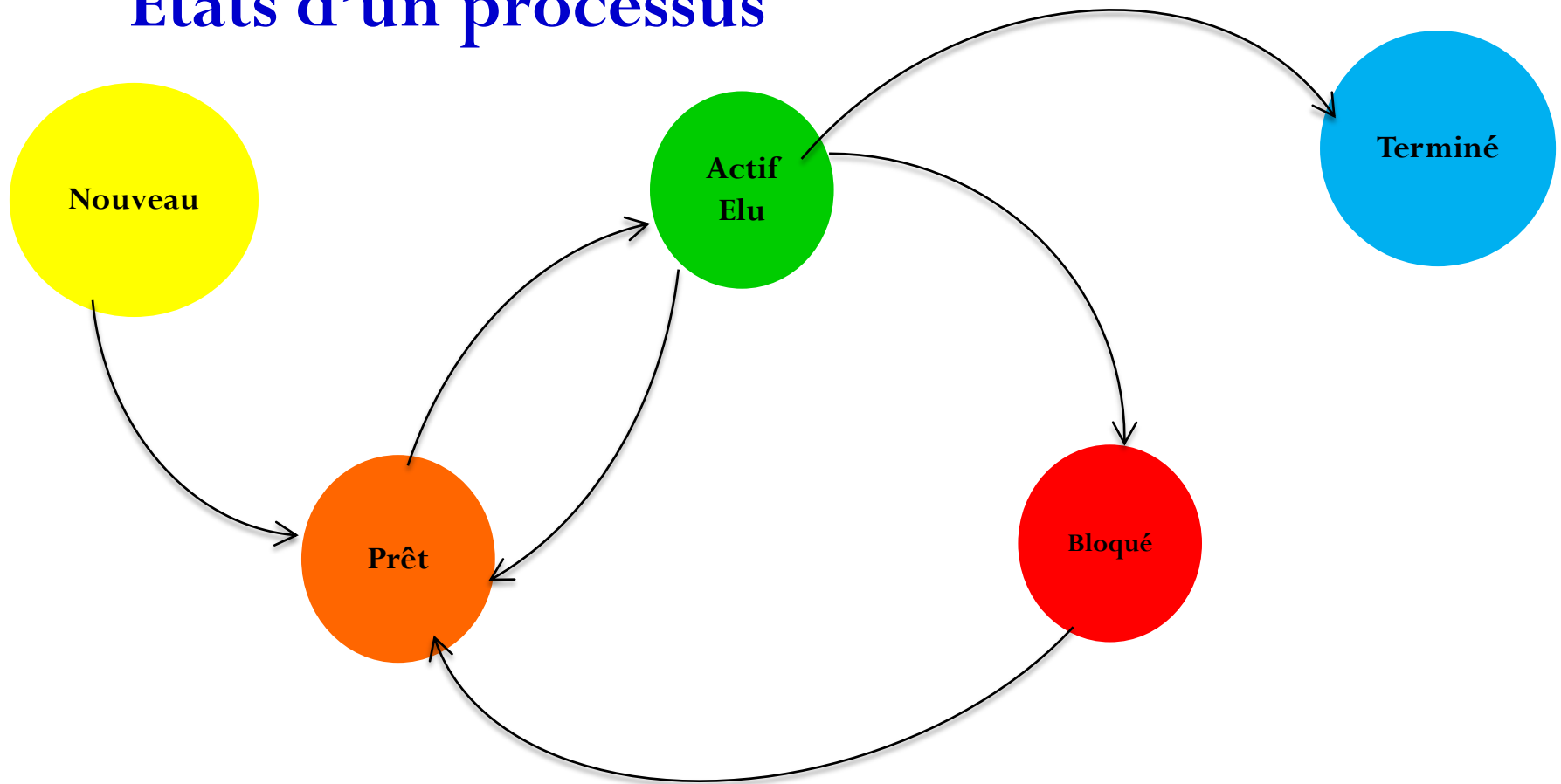
Etats d'un processus (2/3)

•Sémantique des transitions



- ❶ Attribution du processeur au processus sélectionné
- ❷ Réquisition du processeur après expiration de la durée du temps par exemple
- ❸ Blocage du processus actif en attendant Un évènement tel que un E/S
- ❹ Débloquage du processus après disponibilité de l'évènement bloquant

Etats d'un processus



- Nouveau: Le SE a créé le processus
- Terminé: Le processus n'est plus exécutable

Propriétés d'un processus

- A un instant donné, un processus est caractérisé par plusieurs informations :
 - Son état: Actif, suspendu, etc. ;
 - Un numéro d'identification unique (PID) ;
 - Son compteur ordinal : registre qui contient l'adresse mémoire de l'instruction prochainement exécutée;
 - Ses données en mémoire ;
 - Des informations indiquant le processus père, les processus fils, le groupe, les variables d'environnement, les statistiques et les limites d'utilisation des ressources....
 - Toutes autres informations utiles à son exécution (E/S, fichiers ouverts, etc.) ;

Propriétés d'un processus

- Plusieurs identificateurs sont associés à un processus.
- Dès sa création, un processus reçoit les paramètres suivants :
 - PID : Numéro de processus;
 - PPID : Numéro du processus père;
 - UID : Identificateur de l'utilisateur qui a lancé le processus;
 - GID : Identificateur du groupe de l'utilisateur qui a lancé le processus;

Le bloc de contrôle et le contexte d'un processus

- Pour suivre son évolution, le SE maintient pour chaque processus une structure de données particulière appelée **bloc de contrôle de processus** (PCB : Process Control Bloc) et dont le rôle est de reconstituer le contexte du processus.
- Le contexte d'un processus est l'ensemble des informations dynamiques qui représente l'état d'exécution d'un processus.
- Le PCB contient aussi des informations sur l'ordonnancement du processus (priorité du processus, les pointeurs sur les files d'attente).

Ordonnancement des processus

Définition

L'ordonnanceur (scheduler en anglais) est un programme du SE qui s'occupe de choisir, selon une politique (ou algorithme) d'ordonnancement donnée, un processus parmi les processus prêts pour lui affecter le processeur.

Les critères d'évaluation entre les algorithmes d'ordonnancement sont:

- **Utilisation du processeur** : Un bon algorithme d'ordonnancement sera celui qui maintiendra le processeur aussi occupé que possible.
- **Capacité de traitement** : C'est le nombre de processus terminés par unité de temps.
- **Temps d'attente** : C'est le temps passé à attendre dans la file d'attente des processus prêts.
- **Temps de séjour** : C'est le temps passé depuis l'admission (état prêt) jusqu'à la terminaison (état fin).

Ordonnancement des processus

Les algorithmes d'ordonnancement

1. L'algorithme du Premier Arrive Premier Servi (FCFS)
2. L'algorithme du Plus Court d'abord (SJF)
3. Ordonnancement avec priorité
4. L'algorithme de Round Robin (Tourniquet)
5. L'algorithme du plus court temps restant (SRT)

Création et destruction d'un processus

Création de processus

- Créer un processus, consiste à lui attribuer un identificateur et initialiser son état.
- L'identificateur doit être unique pour distinguer sans ambiguïté le processus.
- Un processus qui vient d'être créé peut à son tour donner naissance à d'autres processus. Ce processus est appelé processus père et les descendants constituent des fils.

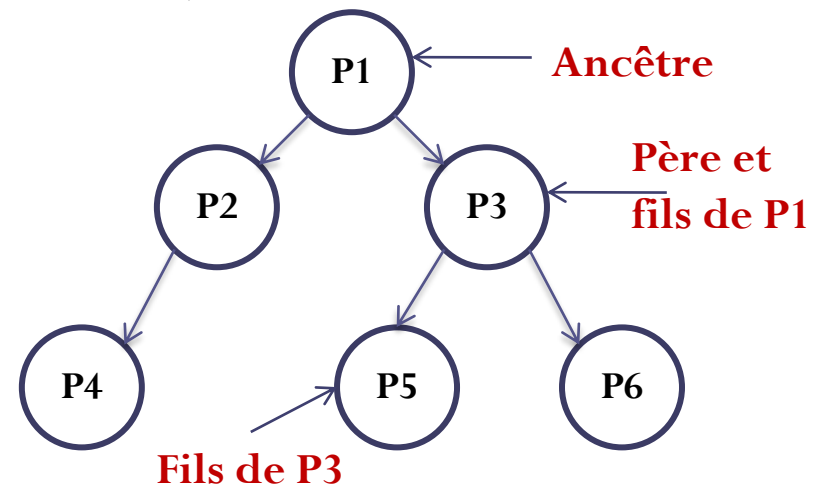


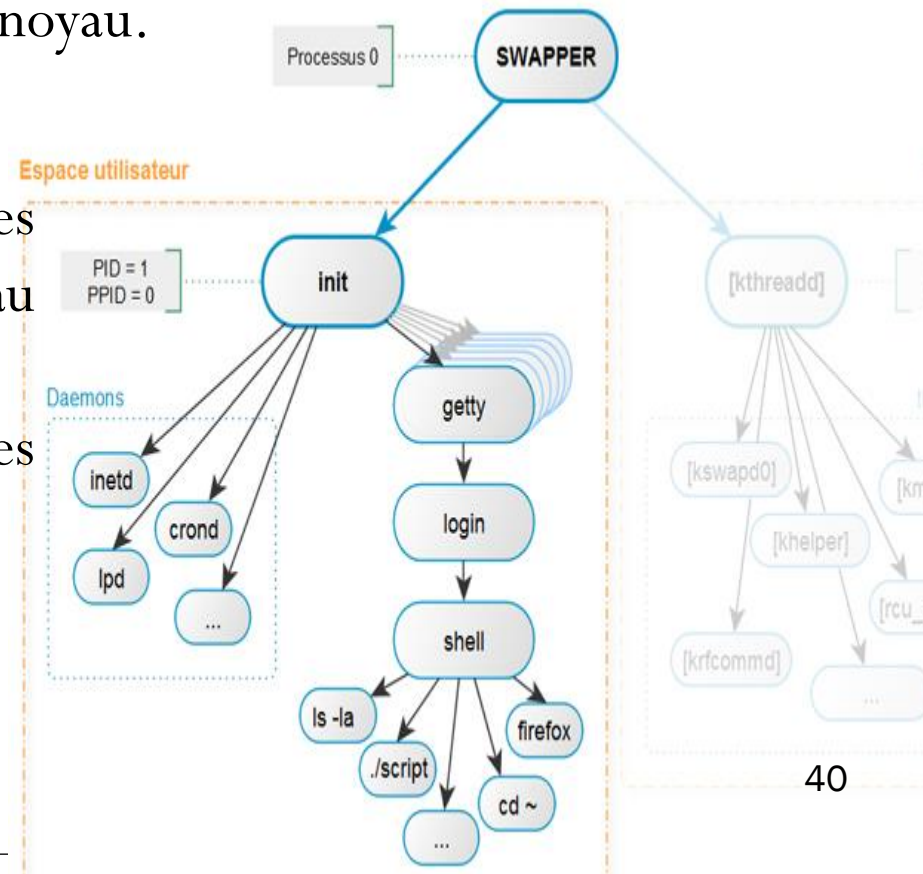
Figure1: L'arbre des processus 39

Création de processus

Lors du démarrage de Unix/Linux, 3 processus sont créés :

1. le Swapper (pid = 0) qui gère la mémoire et crée les 2 autres processus;
2. le Init (pid = 1, ppid=0) qui crée tous les autres processus dans l'espace utilisateur.
3. Kthreadd (pid = 2, ppid=0) à l'espace noyau.

- **Init** se chargera ensuite de créer les autres processus nécessaires au fonctionnement du système.
- **Init** sera ici l'ancêtre de tous les processus.



Destruction de processus

- La destruction d'un processus peut avoir lieu pour deux raisons :
 - Destruction normale (ordinaire) : le processus vient de se terminer d'une façon normale car il a achevé le traitement pour lequel il a été créé.
 - Destruction anormale : le système d'exploitation met fin à un processus sans qu'il termine correctement son objectif car il vient de causer un mauvais fonctionnement.
- La destruction d'un processus conduit à la suppression de ses propriétés et la libération de toutes les ressources qui ont été occupées.

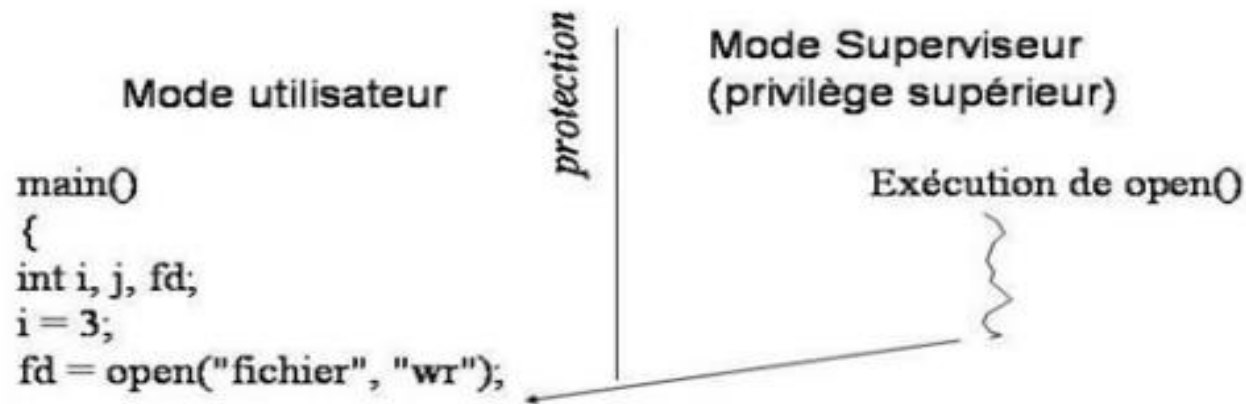
Appels système (1/2)

- **Appel système** est une demande de service c'est-à-dire lorsqu'un processus a besoin d'exécuter une opération spécifique, il demande au SE de la réaliser pour son compte et lui envoyer le résultat
- Lorsqu'une application souhaite accéder à une ressource, ou plus généralement invoquer une fonction du SE, elle procède à un **appel système**.
- Les appels systèmes se présentent donc comme des fonctions/procédures classiques.
 - **open, read, write** et **close** qui permettent les manipulations sur les systèmes de fichiers

Appels système (2/2)

- Exemple:

```
fd = open("mon_fichier.txt", O_WRONLY);  
write(fd, "hello world\n", 12);  
close(fd);
```



Création de processus avec `fork()` sous UNIX

- L'appel système *fork()* est une façon qui permet de créer des processus aussi appelés **processus lourds** :
 - Création par des appels systèmes lents
 - Changement de contexte lent
 - Espace d'adressage propre (pas de partage de la mémoire simple)
 - Copies de toutes les variables et ressources nécessaires (pile, registres, fichiers ouverts...)
 - Mécanismes de communication lourds (pipe, signaux...)
 - ...

Exemple 1 : Instruction de création :

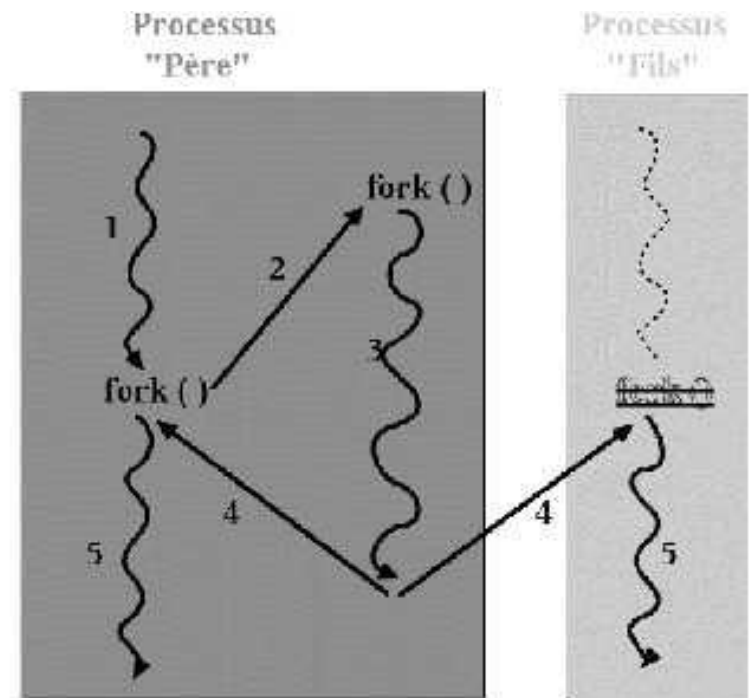
```
#include <unistd.h>
```

```
int fork();
```

- `fork()` est le moyen de créer des processus, par duplication d'un processus existant.
- L'appel système `fork()` crée une copie exacte du processus original ainsi que son contexte initial.

Création de processus avec fork() sous UNIX

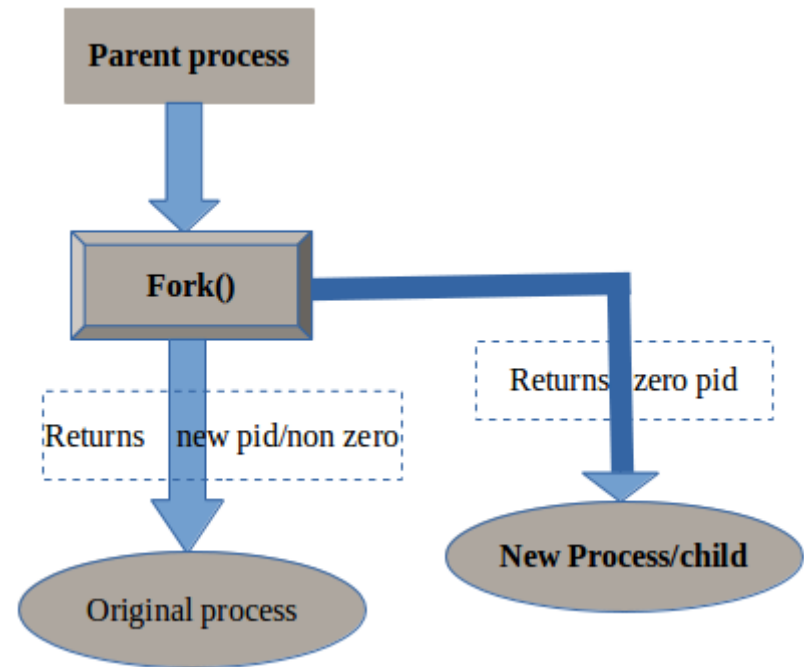
- Les deux processus sont exactement identiques et exécutent le même programme sur **deux copies distinctes** des données. Les espaces de données sont complètement séparés : la modification d'une variable dans l'un est invisible dans l'autre.
- Les processus "père" et "fils" deviennent concurrents (phase n°5) vis-à-vis de l'ordonnanceur. Le processus ainsi créé hérite, du processus qui l'a créé, un certain nombre d'attributs :
 - Le même code,
 - Une copie de la zone de données,
 - L'environnement,
 - La priorité,
 - Les descripteurs de fichiers courants (mêmes caches et mêmes pointeurs),
 - Le traitement des signaux.



Création de processus avec fork() sous UNIX

Pour distinguer le processus père du processus fils on regarde la valeur de retour de fork(), qui peut être :

- La valeur 0 dans le processus fils.
- Positive pour le processus père et qui correspond au PID du processus fils
- Négative si la création de processus a échoué ;



Création de processus avec `fork()` sous UNIX

La fonction **`fork()`** réalise les tâches suivantes :

- 1) Test d'existence de ressources mémoires suffisantes pour permettre la création du nouveau processus,
- 2) Calcul du PID du processus enfant. Le noyau explore ensuite la table `proc[]` pour vérifier que ce numéro n'est pas en cours d'utilisation. Sinon le calcul est relancé,
- 3) Recherche d'une entrée libre dans la table des processus,
- 4) Duplication du contexte processeur et mémoire du processus parent,
- 5) Mise à jour des tables système modifiées par la création du nouveau processus.

Création de processus avec fork() sous UNIX

Puisque les processus créés par fork() s'exécutent de façon concurrente avec le père, on ne peut pas prévoir la politique et l'état de l'ordonnanceur du système,

- Il sera impossible de déterminer quels processus se termineront avant tels autres (y compris leur père).
- Si le père est indisponible à la terminaison de son fils, on dit que le fils est dans un état zombi.

Zombi est un processus qui s'est achevé, mais qui dispose toujours d'un identifiant de processus (PID) et reste donc encore visible dans la table des processus.

- Si un processus père termine avant son fils, le noyau rattache le processus fils (**orphelin**) au processus **init**, d'où l'existence, dans certains cas, d'un problème de synchronisation

Application

Préciser le nombre de processus créés par les programmes ci-dessous puis dessiner l'arbre généalogique des processus engendrés et indiquer les valeurs de variables pour chaque processus :

Code 1	Code 2	Code 3	Code 4	Code 5
<pre>int main() { fork(); fork(); fork(); }</pre>	<pre>int main() { int x,y,z; x=fork(); y=fork(); z=fork(); }</pre>	<pre>int main() { int x=0; int y=0; if (fork() > 0) x=2; else y=4; }</pre>	<pre>int main() { if (fork() > 0) fork(); }</pre>	<pre>int main() { int cpt=0; while (cpt < 3) { if (fork() > 0) cpt++; else cpt=3; }}</pre>

C'est quoi le problème d'un processus ???



Il est un peu lourd, cela implique :

- des recopies de mémoire
- et surtout de passer par le système pour communiquer avec d'autre processus.

Les threads

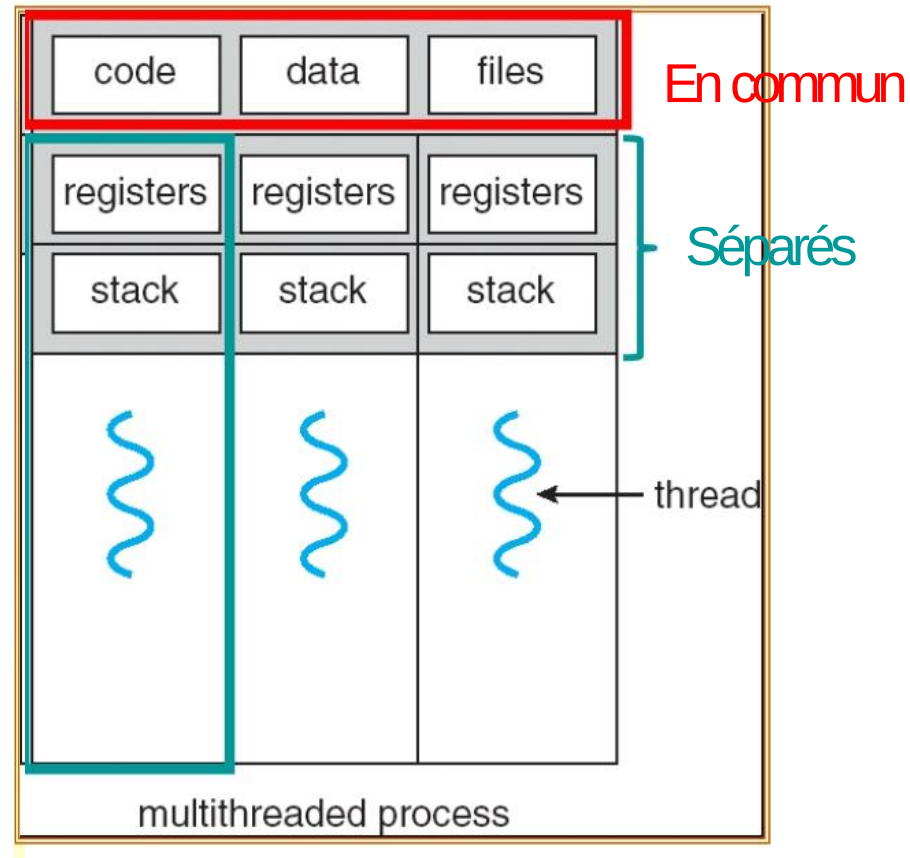
Concept de Thread

- Chaque processus peut fonctionner en lançant des sous-tâches internes au processus. Ces sous-tâches sont nommées, « flot d'exécution », « **processus légers** », ou « **Threads** ».
- De nombreux systèmes d'exploitation modernes offrent la possibilité d'associer à un même processus plusieurs chemins d'exécution (multithreading).
- Un thread, c'est un découpage d'un processus : Il utilise exactement la même mémoire qu'un processus, ainsi que les mêmes descripteurs de fichiers.

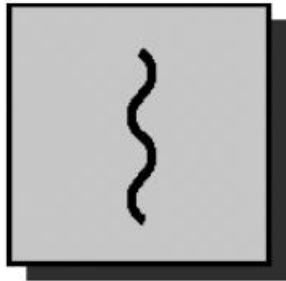
Concept de Thread

- Le processus est vu donc comme étant un ensemble de ressources (code exécutable, segments de données (variable globales et statiques), de fichiers, de périphériques, etc.) que les threads partagent.
 - Ressources propres à chaque processus léger :
 - Les variables locales
 - Les registres
 - La pile d'exécution (stack)
 - Le compteur ordinal
 - Ressources partagées entre les processus légers d'un même processus :
 - Le code
 - **Les variables globales**
 - La table des fichiers ouverts

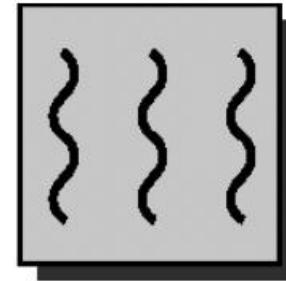
Concept de Thread



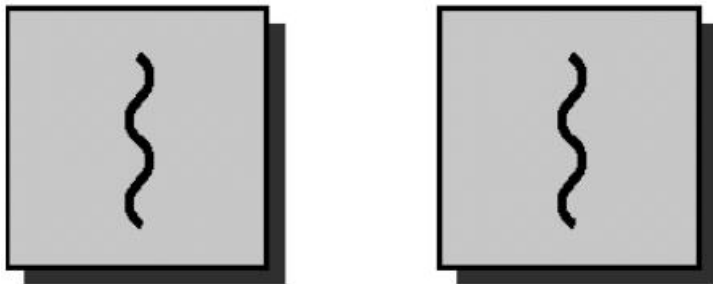
Threads et processus



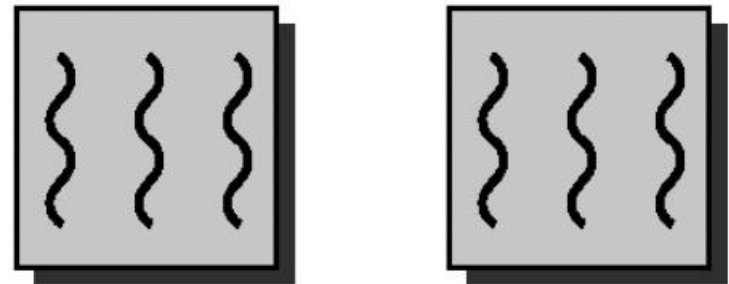
one process
one thread



one process
multiple threads



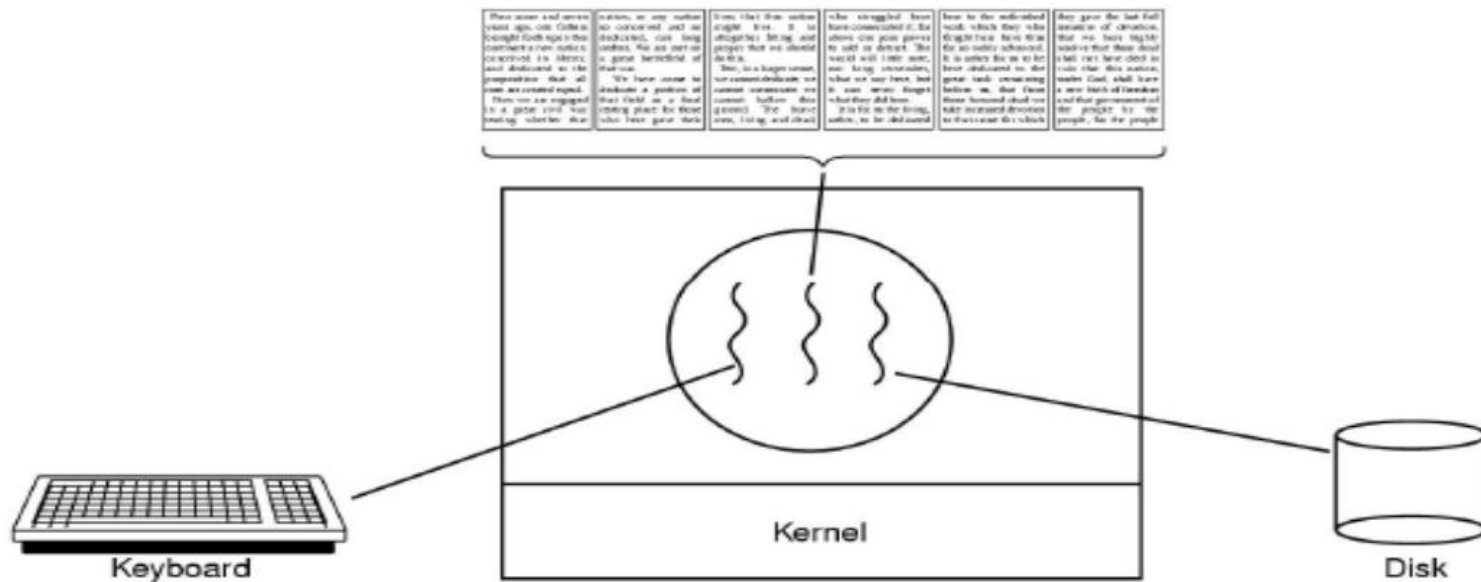
multiple processes
one thread per process



multiple processes
multiple threads per process

Cas d'utilisation de threads

Word processor



Le processus MS-Word peut impliquer plusieurs threads:

Interaction avec le clavier

Rangement de caractères sur la page

Sauvegarde régulière du travail fait

Contrôle d'orthographe

Etc.

Ces threads partagent tous le même fichier .doc et autres données

Avantages des Threads

- **Réactivité** : Un processus peut être subdivisé en plusieurs threads.
 - L'un peut s'exécuter tant que l'autre est bloqué.
- **Partage de ressources.**
- **Économie d'espace mémoire et de temps.**
 - Exemple : Sous Solaris, la création d'un processus est 30 fois plus lente que celle d'un Thread.
- **Moins de charge** pour créer, ordonnancer et détruire un thread.
- **Communication facile entre les Threads**

Types de threads

La majorité des systèmes permettent le multithreading (multiflot). Ils sont offerts soit au niveau utilisateur, soit au niveau noyau :

- Threads utilisateur

- Gérés par une bibliothèque utilisateur (par exemple **pthread** sous Linux ou **thread** dans Solaris)
- Le changement de contexte est rapide.

- Threads noyau

- Ils sont directement supportés par le noyau du système d'exploitation.
- Le système d'exploitation se charge de leur gestion

Service POSIX pour la gestion des threads

- Il définit un ensemble de structure de données et de fonctions C implantant des threads : *pthread*

`#include <pthread.h>`

- Création de threads

```
int pthread_create (pthread_t *thread , pthread_attr_t *attr,  
void *nomfonction, void *arg );
```

- Le service `pthread_create()` crée un processus léger qui exécute la fonction `nomfonction` avec l'argument `arg` et les attributs `attr`. (taille de pile, priorité, etc.) . Elle retourne zéro si le thread est créé sinon un nombre indiquant l'erreur.

Services POSIX pour la gestion des threads

- Suspension des threads

```
int pthread_join(pthread_t *thid, void **valeur_de_retour);
```

- pthread_join() suspend l'exécution d'un processus léger (appelant) jusqu'à ce que termine le processus léger avec l'identificateur *thid* et retourne son résultat dans *valeur_de_retour*. Cette fonction retourne l'état de terminaison du processus léger (zéro en cas de succès).

- Terminaison des threads

```
void pthread_exit(void *valeur_de_retour);
```

- pthread_exit() permet à un processus léger de terminer son exécution, en retournant l'état de terminaison.

Application

Écrire un programme C multi-threadé dans lequel le père déclare une variable globale ***nbr_parts_gateau*** initialisée à 8. Il lance par la suite 4 threads dont chacun cherche à consommer un part chaque seconde. Après chaque consommation, le fils affiche le nombre de parts restants.

Dans le cas où le fils ne trouve plus de morceaux, il affiche le message suivant : "thread x : Gâteau terminé !!!», où x est le numéro du thread correspondant. Le père doit attendre la terminaison de tous ses fils.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <sys/types.h>
```

```
#include <pthread.h>
```

```
volatile int nbr_parts_gateaux=8;
```

```
void* manger (void* args)
```

```
{
```

```
    while (nbr_parts_gateaux > 0)
```

```
    {
```

```
        nbr_parts_gateaux--;
```

```
        printf ("thread%d : nombre de parts restants = %d\n", (int)(long)args, nbr_parts_gateaux);
```

```
        sleep(1);
```

```
    }
```

```
    printf ("thread%d : Gateau terminé\n", (int)(long)args);
```

```
    return 0;
```

```
}
```

```
int main()
```

```
{
```

```
    pthread_t tab[4];
```

```
    int i;
```

```
    for (i=0; i<4; i++)
```

```
        pthread_create(&tab[i], NULL, manger, (void*) i);
```

```
    for (i=0; i<4; i++)
```

```
        pthread_join(tab[i], NULL); /*Eviter le risque de terminer le programme complètement  
sans avoir pu exécuter les threads */
```

```
    return 0;
```

```
}
```

La valeur sera toujours relue en mémoire à chaque accès et non stockée dans un registre temporaire

NULL pour utiliser taille de pile et priorité par défaut

NULL : pas besoin de retour du thread appelé

```

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <pthread.h>
#include <unistd.h> // pour sleep()
volatile int nbr_parts_gateaux = 8; // variable partagée
pthread_mutex_t verrou; // déclaration du mutex

```

```

void* manger(void* args)
{
    int id = (int)(long)args; // conversion sûre
    while (1)
    {
        pthread_mutex_lock(&verrou); // Début section critique
        if (nbr_parts_gateaux > 0)
        {
            nbr_parts_gateaux--;
            printf("thread %d : nombre de parts restantes = %d\n", id, nbr_parts_gateaux);
            pthread_mutex_unlock(&verrou); // Fin section critique
            sleep(1); // simule le temps pour manger une part
        }
        else
        {
            pthread_mutex_unlock(&verrou); // Débloquent avant de sortir
            break; // plus de gâteau
        }
    }
    printf("thread %d : gâteau terminé\n", id);
    return 0;
}

```

```

int main()
{
    pthread_t tab[4];
    int i;
    pthread_mutex_init(&verrou, NULL); // initialisation du mutex

    for (i = 0; i < 4; i++)
        pthread_create(&tab[i], NULL, manger, (void*)(long)i);
    for (i = 0; i < 4; i++)
        pthread_join(tab[i], NULL);
    pthread_mutex_destroy(&verrou); // libération du mutex
    return 0;
}

```

Problèmes avec concurrence = parallélisme

- Les threads concurrents doivent parfois partager des ressources et des données (fichiers ou mémoire commune)
 - On parle donc de threads *coopérants*
- Si l'accès n'est pas contrôlé, le résultat de l'exécution du programme pourra dépendre de l'ordre d'entrelacement de l'exécution des instructions (*non-déterminisme*).
- Un programme pourra donner des résultats différents et parfois indésirables de fois en fois.

Exemple1

Deux threads exécutent cette même procédure et partagent la même base de données.

Ils peuvent être interrompus n'importe où.

Le résultat de l'exécution concurrente de P1 et P2 dépend de l'ordre de leur entrelacement.

**M. X demande une
réservation
d'avion**

**Base de données
dit que fauteuil
A est disponible**

**Fauteuil A est
assigné à X et
marqué occupé**

Exemple1

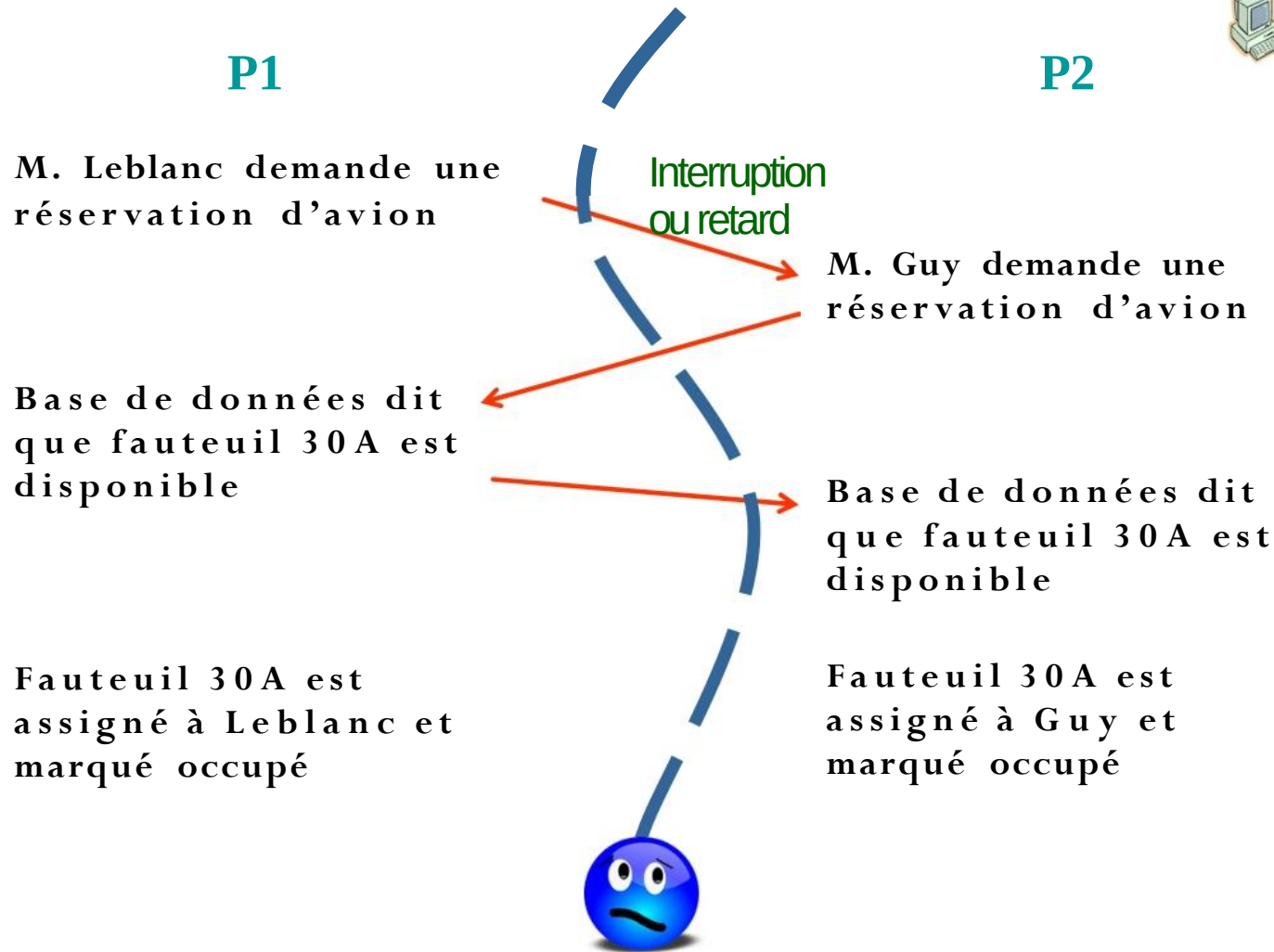


Figure : Vue globale d'une exécution possible

Exemple2

Processus P1

```
int a;  
  
void main()  
{  
....  
    scanf("%d",&a)  
    printf("%d",a)  
}
```

Processus P1

```
int a;  
  
void main()  
{  
....  
    scanf("%d",&a)  
    printf("%d",a)  
}
```

- Si la var a est partagée, le premier a est effacé.
- Si elle est privée, l'ordre d'affichage est renversé

Asynchronie des threads

- Quand plusieurs threads exécutent en parallèle, nous ne pouvons pas faire d'hypothèses sur la vitesse d'exécution des threads, ni leur entrelacement.
- Le partage d'objets sans précaution particulière peut conduire à des résultats imprévisibles (incohérents).
- La solution à ce problème s'appelle **synchronisation des processus**.

Synchronisation interprocessus

- Problème de l'exclusion mutuelle
- Synchronisation
 - o Solutions matérielles
 - o Solutions logicielles

Définitions

- **La synchronisation** : C'est le contrôle de l'ordre d'exécution des tâches
- **Ressources critiques** :
 - Les ressources partagées, comme des zones mémoire, périphériques d'entrées/sorties, ..., sont dites critiques si elles ne peuvent être utilisées simultanément que par un seul processus.
- **Exemple** : Deux processus P et Q produisent des données devant être imprimées sur une imprimante unique. L'emploi de cette imprimante par P exclut l'emploi de l'imprimante par Q tant que l'impression par P n'est pas terminée
 - L'imprimante est une ressource critique : Un seul point d'accès

Définitions

- Section critique
 - Une SC est une suite d'instructions qui opèrent sur une ou plusieurs ressources partagées et qui nécessitent une utilisation exclusive de ces ressources (ressources critiques).
 - Une section critique (SC) est un ensemble d'instructions d'un programme qui peuvent engendrer des résultats imprévisibles (ou incohérents) lorsqu'elles sont exécutées simultanément par des processus différents.

Interblocage (Deadlock)

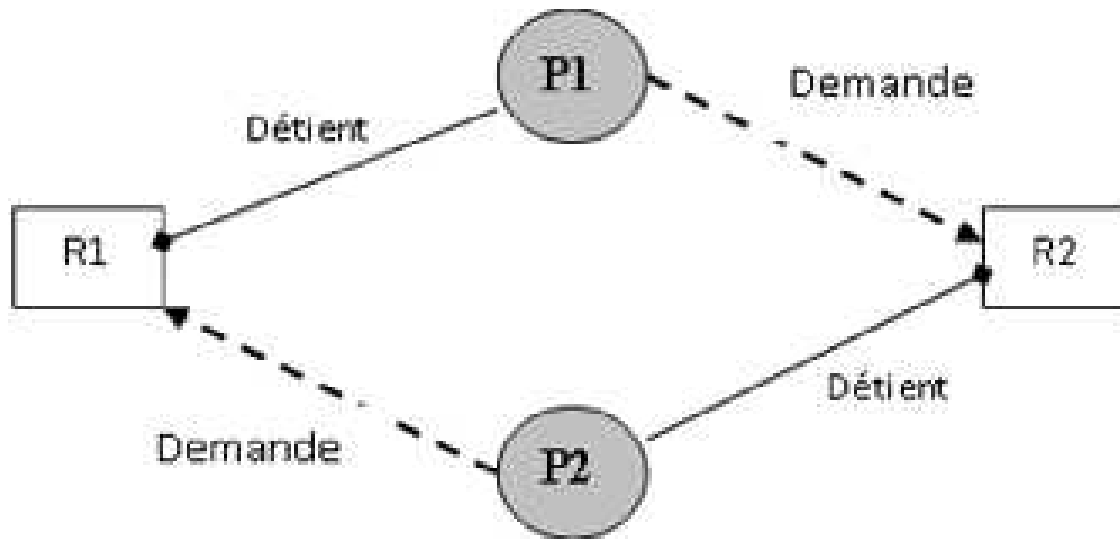
- Un ensemble de processus est dans une situation d'interblocage si chaque processus de l'ensemble attend un événement qui ne peut être produit que par un autre processus de l'ensemble.
- Autrement, l'interblocage est vu comme étant un état anormal où, au moins, deux processus se trouvent incapables de poursuivre leurs exécutions. Cet état est caractérisé par le fait que chaque processus est bloqué et ne peut être débloqué que par un autre processus de l'ensemble.

Interblocage (Deadlock)

- Une situation d'interblocage peut survenir si les quatre conditions suivantes se produisent simultanément :
 - 1- Accès exclusif : Les ressources ne peuvent être exploitées que par un seul processus à la fois.
 - 2- Attente et occupation : Les processus qui demandent de nouvelles ressources gardent celles qu'ils ont déjà acquises et attendent la satisfaction de leur demande
 - 3- Pas de réquisition : Les ressources déjà allouées ne peuvent pas être préemptées.
 - 4- Attente circulaire : Les processus en attente des ressources déjà allouées forment une chaîne circulaire d'attente.

Exemple d'interblocage

Un système possède une instance unique de chacun des deux types de ressources R1 et R2. Un processus P1 détient l'instance de la ressource R1 et un autre processus P2 détient l'instance de la ressource R2. Pour suivre son exécution, P1 a besoin de l'instance de la ressource R2, et inversement P2 a besoin de l'instance de la ressource R1. Une telle situation est une situation d'interblocage.



Exclusion mutuelle

- Pour assurer l'exécution des sections critiques des processus, il est nécessaire de trouver une solution qui permet d'éviter les conditions de concurrence.
- L'exclusion mutuelle est une méthode qui permet de s'assurer que si un processus utilise une variable ou une ressource partagée, les autres processus seront exclus de la même activité.

Exclusion mutuelle

- Une instruction de type exclusion mutuelle se décompose en 3 étapes

Section non critique.

Protocole d'entrée en section critique.

Section critique.

Protocole de sortie de la section critique.

Section non critique.

- Les deux protocoles d'entrée et de sortie permettent de contrôler l'obtention et la libération du droit d'accès en exclusion mutuelle par un processus donné
- Protocole d'entrée en SC: ensemble d'instructions qui permet cette vérification et la non progression éventuelle;
- Protocole de sortie de SC: ensemble d'instructions qui permet à un processus ayant terminé sa SC d'avertir d'autres processus en attente que la voie est libre

Principe de l'exclusion mutuelle

Certaines conditions doivent être vérifiées pour toute solution de synchronisation :

1. **Exclusion mutuelle** : A tout instant, un processus au plus peut se trouver en section critique
2. **Pas de famine**
3. **Pas d'interblocage** : Si un processus est bloqué hors sa section critique, ce blocage ne doit pas empêcher l'entrée d'un autre processus en sa section critique
4. **Uniformité** : La solution doit être la même pour tous les processus càd, qu'aucun processus ne doit jouer le rôle privilégié

=> **Comment assurer alors une coopération correcte et efficace des processus ?**

Solution pour assurer l'exclusion mutuelle

- Solutions de niveau Matériel :
 - Masquage des interruptions
 - Instruction indivisible, TAS (Test and Set)
- Solutions logicielles :
 - Alternance, Peterson
 - Dijkstra (Semaphore)

Masquage des interruptions

- Protocole d'entrée en SC : masquer les interruptions au niveau du CPU (interruption d'horloge qui permet d'interrompre un processus lorsqu'il a épuisé son quantum).
- La SC ne peut pas être interrompue car les changements de contexte sont impossibles.
- Protocole de sortie de SC : démasquer les interruptions.

Masquer les IT
SC
Démasquer les IT ;

Masquage des interruptions

Remarques :

- Le masquage des IT n'est accessible qu'aux programmeurs privilégiés pour des raisons de fiabilité comme le super-utilisateur.
- Ne convient pas pour les machines multi-processeurs à mémoire partagée (car chaque processeur gère ses propres interruptions).

Analyse :

- Absence de blocage : l'ordonnanceur du CPU finit par donner la main à un processus voulant rentrer dans sa SC.
- Condition de progression : une tâche doit pouvoir entrer en SC si aucune autre ne s'y trouve.
- Absence de famine : si l'ordonnanceur l'assure (normalement oui !).
- **Solution dangereuse en mode utilisateur :**
 - Si dans un processus, le programmeur a oublié de restaurer les interruptions, c'est la fin du système ☹

Instruction indivisible, TAS (Test and Set)

- Les ordinateurs multiprocesseurs ont une instruction atomique indivisible appelée **TAS** (Test and Set Lock).
- Lorsqu'un processeur exécute l'instruction **TAS**, il **verrouille le bus de données** pour empêcher les autres processus d'accéder à la mémoire pendant la durée de l'opération.

Si un processus est entrain d'effectuer un test-and-set sur un emplacement mémoire

Si plusieurs processus tentent d'accéder à la même mémoire



Aucun autre processus ne peut commencer un autre test-and-set jusqu'à ce que le premier processus soit terminé

TAS : Protéger l'espace mémoire en cas d'accès concurrents

Instruction indivisible, TAS (Test and Set)

- Cette instruction peut être utilisée pour assurer l'exclusion mutuelle.
- Voilà d'une façon simplifier l'esprit de cette méthode :

verrou: variable commune, initialement 0 (« déverrouillé »)

Si verrou = 0 alors
TAS met verrou à 1
et retourne **0** dans
la variable prev
Sinon TAS retourne
la val de verrou (1)

P1

```
prev : var locale
...
prev = TAS(verrou);
Tant que (prev = 1)
Faire
    /* sleep(délai); */
    prev = TAS(verrou);
Fintantque
    <section critique>
    verrou = 0;
...
```

P2

```
prev : var locale
...
prev = TAS(verrou);
Tant que (prev = 1)
Faire
    /* sleep(délai); */
    prev = TAS(verrou);
Fintantque
    <section critique>
    verrou = 0;
...
```

- Cette solution est acceptable dans un système multiprocesseur et si l'exclusion mutuelle survient rarement.

Alternance

- Utiliser une variable tour qui mémorise le tour du processus qui doit entrer en section critique. Tour est initialisée à 0.

- Protocole d'entrée en SC :
while (tour != monNumero) ; // attente active
- Protocole de sortie de la SC :
tour=(monNumero +1) % N ; // au suivant !

N est le nombre de processus qui souhaite exécuter la SC

Alternance

Rien faire

Thread T0:

`while(tour!=0);`

`// premier à entrer`

`SC`

`tour = 1;`

`nonSC`

`while(tour!=0);`

`// entre quand T1 finit`

`SC`

`tour = 1;`

`nonSC`

`etc...`

Thread T1:

`while(tour!=1);`

`// entre quand T0 finit`

`SC`

`tour = 0;`

`nonSC`

`while(tour!=1);`

`// entre quand T0 finit`

`SC`

`tour = 0;`

`nonSC`

`etc...`

Alternance

- **Avantage :** simple et facile à utiliser
- **Inconvénients :**
 - Problème de famine, un processus possédant tour, peut ne pas être intéressé immédiatement par la section critique
 - Interblocage si le tour est cédé à un processus à l'état bloqué

Peterson (1981)

- Combine les deux idées : veut[i]=intention d'entrer; tour=à qui le tour
- Initialisation : veut[0] = veut[1] = faux et tour = i ou j
- Désir d'exécuter SC est indiqué par veut[i] = vrai

Prot. d'entrée {
Thread T0: while(true){
 veut[0] = vrai;
 // T0 veut entrer
 tour = 1;
 // T0 donne une chance à T1
SC
nonSC
}

Thread T1: while(true){
 veut[1] = vrai;
 // T1 veut entrer
 tour = 0;
 // T1 donne une chance à 0
SC
nonSC
}

Peterson (1981)

- **Avantage : Ni famine ni interblocage**
 - Un processus souhaitant utiliser la SC doit le signaler et positionner soit même la variable tour
 - Un processus intéressé peut entrer en SC même si c'est pas son tour quand l'autre ne l'est pas
- **Inconvénients :**
 - Attente active
 - Processus prioritaire est mis en attente si l'autre est intéressé et a le tour

Attente active/ Attente passive

- L'attente peut être :
 - **Active** : le protocole d'entrée à la SC est une boucle dont la condition est un test qui porte sur des variables indiquant la présence ou non d'un processus en section critique.
 - **Non active** : le processus passe dans l'état endormi et ne sera réveillé que lorsqu'il sera autorisé à entrer en section critique.
- Les solutions avec attente active sont gourmandes en temps CPU

Solutions évitant l'attente active...

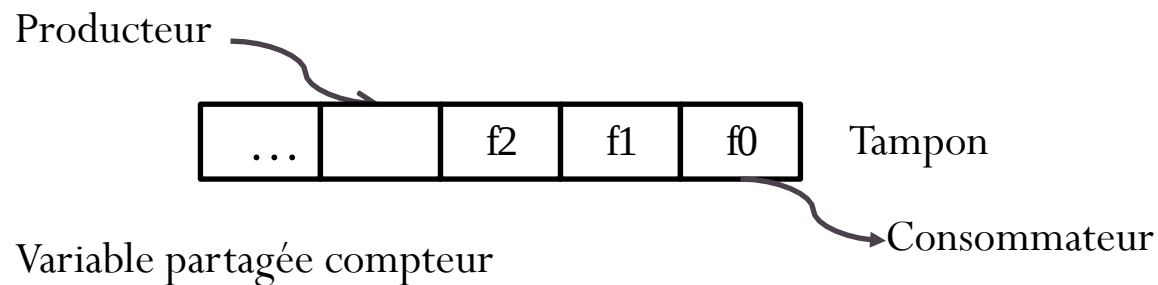


Synchronisation sans attente active

- L'idée est qu'un processus qui ne peut pas entrer en SC passe à l'état bloqué au lieu de consommer le temps processeur inutilement. Il sera réveillé lorsqu'il pourra y entrer.
- Solutions :
 - Appels système **sleep et wakeup**
 - **Sémaphores**
 - **Moniteurs**
 - Etc.

Application des primitives Sleep et Wakeup au modèle Producteur Consommateur

- Le système d'exploitation offre deux appels système:
 - **Sleep** (dormir) qui bloque le processus appelant.
 - **Wakeup** (réveiller) qui réveille le processus donné en argument.
- Producteur/Consommateur: Deux processus (le producteur et le consommateur) coopèrent en partageant un même tampon :
 - Le producteur produit des objets qu'il dépose dans le tampon.
 - Le consommateur retire des objets du tampon pour les consommer.



Application des primitives Sleep et Wakeup au modèle Producteur Consommateur

```
#define N 100      /* taille du tampon */  
int compteur = 0 ; /* nbre d'objets dans tampon */
```

```
void producteur () {  
    while(TRUE)  
    {  
        produire_objet();  
  
        mettre_objet();  
        compteur = compteur + 1;  
  
    }  
}
```

```
Void consommateur () {  
    while(TRUE)  
    {  
  
        retirer_objet();  
        compteur = compteur - 1;  
  
        consommer_objet(...);  
    }  
}
```

Application des primitives Sleep et Wakeup au modèle Producteur Consommateur

Inconvénients :

- L'accès à la variable partagée compteur n'est pas protégé, ce qui peut entraîner des incohérences dans les valeurs prises par cette variable.
- Un signal wakeup envoyé à un processus qui ne dort pas (encore) est perdu.
- Considérer le scénario où une préemption a lieu entre le if et le sleep du producteur, par exemple.

Solutions ...



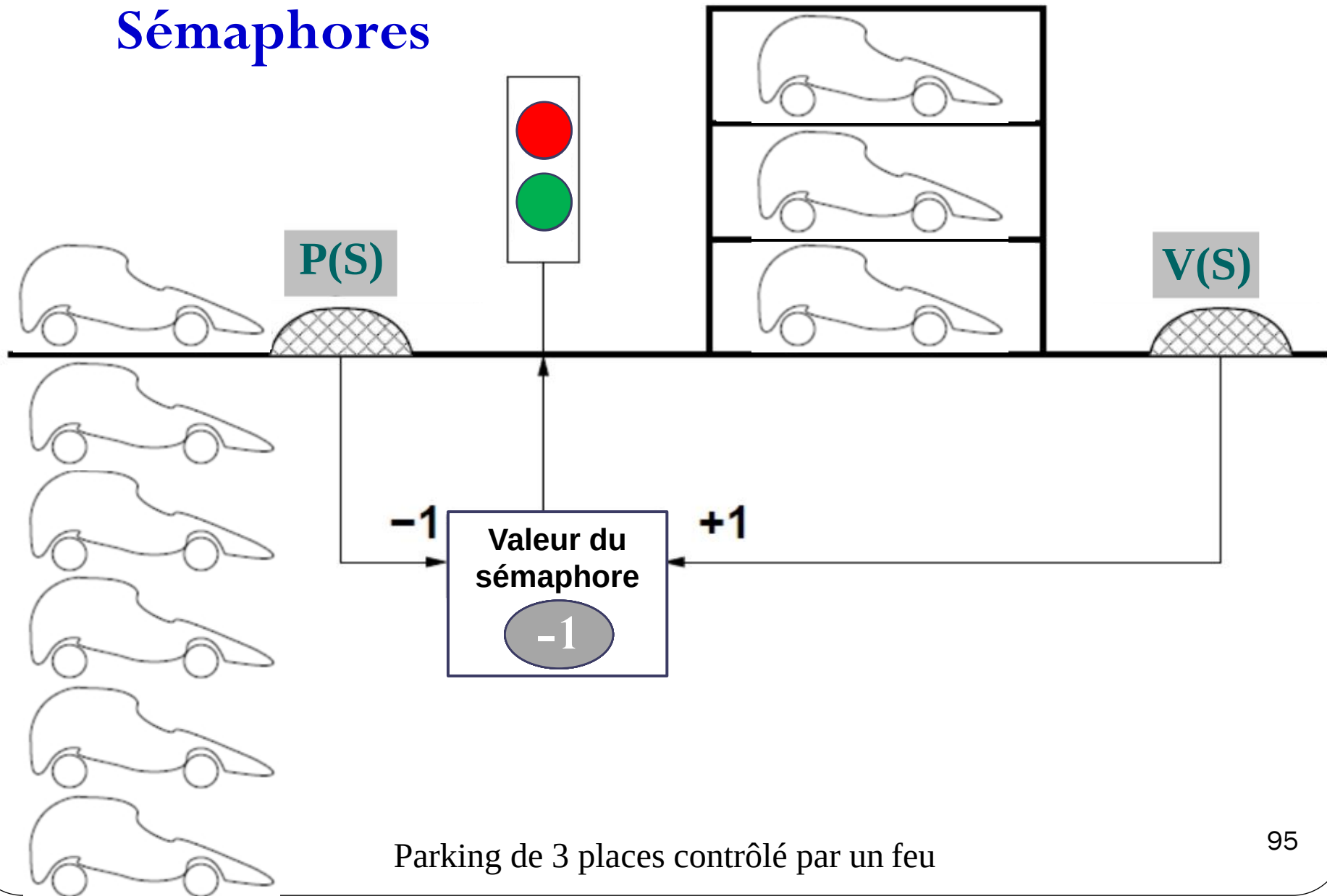
Sémaphores (Dijkstra, 1965)

- Sémaphore est un mécanisme de synchronisation de processus inventé par le physicien et informaticien hollandais Edsger Dijkstra en 1965. Il s'agit d'une structure de données qui comprend :
 - (1) Une variable entière donnant le nombre de ressources disponibles
 - (2) Une file d'attente de processus.
- On manipule un sémaphore uniquement au travers de trois opérations qui ont la particularité d'être **atomiques**, c'est à dire qu'elles s'exécuteront jusqu'à terminaison sans être interrompues par un autre processus :
Init(semaphore sem, int nombre_de_ressources);
P(semaphore sem);
V(semaphore sem);
- **Remarque** : D'autres auteurs les appellent wait et signal ou acquire et release

Sémaphores (Dijkstra, 1965)

- Init(sem, nombre de ressources) est la procédure d'initialisation du sémaphore : le nombre de ressources disponibles de sem est initialisé à *nombre_de_ressources* positif et la file de processus de *sem* est vide. Cette opération n'est effectuée qu'une et une seule fois.
- P(sem) (Proberen:essayer) met le processus dans la file d'attente si le nombre de ressources du sémaphore sem est $\leq$ à 0 et décrémente le nombre de ressources.
- V(sem) (Verhogen:libérer) incrémente le nombre de ressources du sémaphore sem et réveille un processus s'il y en a en attente.

Sémaphores



Parking de 3 places contrôlé par un feu

Sémaphore binaire (Mutex)

- Les sémaphores binaires permettent de réaliser des **exclusions mutuelles**
- Exclusion mutuelle = 1 seule permission accordée
 - Variable commune : Mutex: sémaphore
 - Avant de commencer, une entité exécute Init (Mutex, 1)
 - $\text{Mutex.Val} = 1 \rightarrow$ SC libre
 - $\text{Mutex.Val} = 0 \rightarrow$ SC occupé

```
Init(mutex , 1 )      /* nombre de processus autorisés à entrer
                        simultanément dans la section critique */
P(mutex)
<section_critique>
V(mutex)
```

Modèle producteur/consommateur

Objectif

- Contrôler le flux entre un (ou des) producteur(s) et un (ou des) consommateur(s) dans le cas où ils communiquent via une mémoire tampon de N cases.

1. Exécution Produc : il produit info0

info0				
-------	--	--	--	--

2. Exécution Produc : il produit info1

info0	info1			
-------	-------	--	--	--

3. Exécution Conso : il consomme info0

	info1			
--	-------	--	--	--

4. Exécution Produc : il produit info2

	info1	info2		
--	-------	-------	--	--

Modèle producteur/consommateur

Solution de base

- Ceux qui veulent en retirer des éléments, doivent avant tout savoir s'il y en a à retirer

➡ Un sémaphore n_plein qui dit combien d'éléments disponibles :
Consommateur peut entrer si et seulement s'il y en a

- Ceux qui veulent y apporter des éléments, doivent avant tout savoir s'il y a de l'espace

➡ Un sémaphore n_vide dit combien d'espace dispo : Producteur peut entrer si et seulement s'il y en a

➡ Deux sémaphores sont nécessaires : n_plein et n_vide

Modèle producteur/consommateur

Implémentation

On introduit deux variables dans ce modèle :

- ❑ **n_plein** : nombre des messages occupant le tampon (initialement $n_plein=0$).
- ❑ **n_vide** : nombre d'emplacement libre dans le tampon (initialement $n_vide = N$)
- Le programme principale **main** initialise les deux sémaphores et permet de créer plusieurs processus **prod** et **cons** :

Init(n_vide, N) et Init($n_plein, 0$)

Modèle producteur/consommateur

Init(n_vide,N) et Init(n_plein,0)

Producteur Prod

...

P (n_vide) // *bloquer* producteurs
 déposer (message) // Section Critique

V (n_plein) // *veiller* consommateur

...

Consommateur Cons

...

P (n_plein) // *bloquer* consommateurs
 retirer (message) // Section Critique

V (n_vide) // *veiller* producteur

...



Problème : A un instant on peut avoir N producteurs ou Consommateurs en section critique



Ajouter un sémaphore mutex pour protéger «déposer (message) » et « prélever (message) »

Modèle producteur/consommateur

- On ajoute au programme principale Init(mutex, 1), pour garantir qu'un seul processus n'exécute la section critique.

Producteur Prod

...

P (n_vide) // *bloquer* producteurs

P(mutex)

déposer (message) // Section Critique

V(mutex)

V (n_plein) // *veiller* consommateur

...

Consommateur Cons

...

P (n_plein) // *bloquer* consommateurs

P(mutex)

retirer (message) // Section Critique

V(mutex)

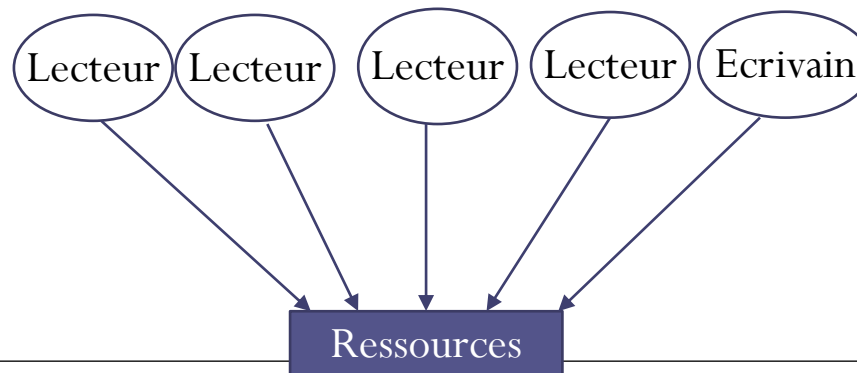
V (n_vide) // *veiller* producteur

...

Problème des rédacteurs et des lecteurs

Objectif

- Permettre une compétition cohérente entre deux types de processus (les « lecteurs » et les « rédacteurs ») sous les contraintes suivantes :
 - ❑ Plusieurs lecteurs peuvent accéder simultanément à la ressource (Base de Données)
 - ❑ Les rédacteurs sont exclusifs entre eux pour leur exploitation de la ressource
 - ❑ Un rédacteur est exclusif avec les lecteurs



Problème des rédacteurs et des lecteurs

Solution de base

- Sémaphore *Redact* initialisé à 1
- Entier *NL* initialisé à 0 et partagé entre les lecteurs
- Sémaphore *mutexL* initialisé à 1

Lecteur

```
P(mutexL)  
NL = NL + 1  
Si NL == 1 alors  
    P(Redact) //Premier lecteur empêche écriture  
Fin  
V(mutexL)  
lectures()  
P(mutexL)  
NL = NL - 1  
Si NL == 0 alors  
    V(Redact) // Dernier lecteur habilite l'écriture  
finSi  
V(mutexL)
```

Rédacteur

```
P (Redact)  
    écriture () //Section Critique  
V(Redact)
```

Problème avec sémaphores

- P et V sont dispersés parmi plusieurs processus, mais ils doivent se correspondre en particulier le cas d'un processus qui a des P et V dans des boucles et des tests...
- Utilisation doit être correcte dans tous les processus : Un seul “mauvais” processus peut faire échouer toute une collection de processus (p.ex. oublie de faire V)

➡ Les Sémaphores gèrent bien l'exclusion mutuelle, mais en cas de problème de programmation ou de synchronisation : **risques d'interblocage**

Les moniteurs

- Les **moniteurs** sont proposés comme une structure de synchronisation de plus haut niveau pour faciliter l'écriture de programmes corrects.
- Ils simplifient la mise en place de sections critiques
- Ils sont définis par :
 - des données internes (appelées aussi variables d'état)
 - des primitives d'accès aux moniteurs (points d'entrée)
 - **wait** : qui libère l'accès au moniteur et bloque le processus appelant sur une condition
 - **signal** : qui réveille sur une condition un des processus en attente à l'intérieur du moniteur (un wait sur la même condition)
 - Des primitives internes (uniquement accessibles depuis l'intérieur du moniteur)
 - Une ou plusieurs files d'attente

Les moniteurs

- Pour assurer l'exclusion mutuelle, à tout moment, un processus au plus est actif dans le moniteur
- Si un processus est actif dans le moniteur et un autre demande à y accéder (appel d'une procédure du moniteur), ce dernier est mis en attente jusqu'à ce que le processus actif dans le moniteur se bloque en exécutant un wait sur une variable conditionnelle (wait(c)) ou quitte le moniteur.
- Le processus bloqué est réveillé par un autre processus en lui envoyant un signal sur la variable conditionnelle (signal(c)).

Différence Sémaphore / Moniteur

Base de comparaison	Sémaphore	Moniteur
De base	Sémaphores est une variable entière S.	Monitor est un type de données abstrait.
action	La valeur de Semaphore S indique le nombre de ressources partagées disponibles dans le système.	Le type de moniteur contient des variables partagées et l'ensemble des procédures qui agissent sur la variable partagée.
Accès	Lorsqu'un processus accède aux ressources partagées, il effectue l'opération wait () sur S et lorsqu'il libère les ressources partagées, il effectue l'opération signal () sur S.	Lorsqu'un processus souhaite accéder aux variables partagées dans le moniteur, il doit y accéder via les procédures.
Variable de condition	Le sémaphore n'a pas de variables de condition.	Le moniteur a des variables de condition.

Les moniteurs

- **Avantages**

- Solution plus simple que les sémaphores puisque le programmeur n'a pas à se préoccuper de contrôler l'accès aux sections critiques

- **Inconvénient**

- La majorité des compilateurs utilisés actuellement ne supportent pas les moniteurs à l'exception de JAVA

Producteur/consommateur version moniteur

Les sections critiques du problème du producteur et du consommateur sont les opérations de dépôt et de retrait dans le tampon partagé. Le dépôt est possible si le tampon n'est pas plein. Le retrait est possible si le tampon n'est pas vide

Producteur/consommateur version moniteur

Moniteur Producteur_Consummateur

```
{
bool nplein, nvide ; //variable conditionnelle pour non plein et non vide
int compteur=0, ic=0, ip=0 ;
void mettre (int objet) { // section critique pour le dépôt
if (compteur==N) wait(nplein) ; //attendre jusqu'à ce que le tampon devienne non
plein
tampon[ip] = objet ; // déposer un objet dans le tampon
ip = (ip+1)%N ;
compteur++ ;
// si le tampon était vide avant le dépôt, envoyer un signal pour réveiller le
consommateur.
if (compteur==1) signal(nvide) ; }
void retirer (int* objet) { // section critique pour le retrait
if (compteur ==0) wait(nvide) ;
objet = tampon[ic] ;
compteur -- ;
if(compteur==N-1) signal(nplein) ; }
}
```